

Contents

1. Introduction
 - 1.1 About KOI
 - 1.2 What is in this content pack
 - 1.3 Pack capabilities at a glance
 - 1.4 What's new
2. Prerequisites
3. Create Your KOI API Key
4. Install the Pack
5. Configure the Integration Instance
 - 5.1 Connection parameters
 - 5.2 Event-collection parameters (Cortex XSIAM only)
6. Verify the Setup
7. Using the Commands
 - 7.1 Devices & organization
 - 7.2 Inventory
 - 7.3 Governance & policies
 - 7.4 Allowlist & blocklist
 - 7.5 Threat intelligence — Koidex catalog
 - 7.6 Collector & diagnostics
 - 7.7 Worked examples
8. Event Collection into Cortex XSIAM
 - 8.1 Useful XQL starting points
9. KOI Alerts Dashboard
10. Koi Unified Script Runner Playbooks
 - 10.1 Architecture
 - 10.2 What must already exist on the tenant (not shipped in the pack)
 - 10.3 Configuration — the "Koi Script Runner" List
 - 10.4 How scripts are matched to endpoints
 - 10.5 Job setup
 - 10.6 Run outcomes
11. Alert Triage, Investigation & Response Playbooks
 - 11.1 The playbooks
 - 11.2 Triage flow and verdicts
 - 11.3 What the item investigation gathers
 - 11.4 What the device investigation gathers
 - 11.5 Response safety
 - 11.6 Attaching the triage
12. Deploying the Full Pack as a Single Object (demisto-sdk)
 - 12.1 Choosing an install path
 - 12.2 One-time setup
 - 12.3 Build and push
 - 12.4 Notes verified in practice
13. Onboarding Individual Items Manually
 - 13.1 Item-by-item reference
 - 13.2 Running only the Script Runner Job
14. Validating That Everything Works
15. Troubleshooting
16. Support & References

Document	KOI Content Pack — Customer User Guide
Customer	[CUSTOMER NAME]
Pack version	1.3.0
Applies to	Cortex XSIAM (event collection + commands) and Cortex XSOAR (commands)
Last updated	July 20, 2026

1. Introduction

1.1 About KOI

KOI is an endpoint security platform that provides visibility and control over the software your workforce actually runs: browser extensions, IDE plugins, code and OS packages, desktop applications, and the emerging agentic AI surface — MCP servers, AI models, skills, and AI-powered tools. KOI discovers these items across your endpoints, scores their risk, enforces allow/block policies, and generates alerts and audit events.

1.2 What is in this content pack

Component	Purpose
KOI integration	Event collector (Alerts + Audit logs into Cortex XSIAM) plus 26 commands for inventory, devices, governance, allow/block lists, and the Koidex catalog.
Koi Parsing Rule	Normalizes raw KOI events arriving in the koi_koi_raw dataset and promotes alert and audit fields for modeling.
Koi Modeling Rule	Maps KOI alert and audit events to the Cortex Data Model (XDM).
Koi Alerts Dashboard	Ready-made XSIAM dashboard visualizing KOI alert activity.
Koi Unified Script Runner playbooks	Three playbooks that run KOI script packages on Cortex-Agent endpoints on a schedule, driven by a Job and configured through a JSON List (section 10).
Alert triage, investigation & response playbooks	Seven playbooks that triage each KOI alert, run a full item and device investigation, compute a verdict, and drive analyst-gated response (section 11).

1.3 Pack capabilities at a glance

Capability	What you get
Telemetry collection	KOI Alerts and Audit logs stream into the koi_koi_raw dataset, normalized by the parsing rule and mapped to the Cortex Data Model by the modeling rule — queryable in XQL and visualized by the bundled dashboard.
Asset & posture visibility	Query the software your workforce actually runs — browser extensions, IDE plugins, npm/PyPI packages, desktop applications, MCP servers and other agentic-AI assets — per item or per device, with KOI's risk scoring.
Threat intelligence	The Koidex catalog returns an independent risk score, findings, compliance data and an AI-generated risk summary for any item, whether or not it is installed in your organization.
Automated alert triage	Every KOI alert is parsed, investigated and classified Benign / Suspicious / Malicious from KOI's own alert type and risk level plus the independent catalog risk. Benign alerts auto-close; the rest escalate for review.
Deep investigation	Per item: catalog risk, AI summary, organization exposure (installs, publisher, signing, endpoint count), the endpoints and users affected, governance state, and remediation/approval history. Per device: everything installed on the affected host and which of it is risky.
Analyst-gated response	A Malicious verdict opens a block approval showing the full investigation; the organization-wide blocklist write happens only after a human approves. Items already on the blocklist short-circuit.
Proactive hunting	A scheduled MCP-server audit flags risky agentic-AI assets across the estate without waiting for an alert.
Fleet script execution	Run KOI script packages (for example the deployment script) on Cortex-Agent endpoints on a schedule, targeted by OS, endpoint group or hostname, configured entirely through a JSON List.

1.4 What's new

- **Alert triage, investigation and response playbooks** — automated triage with a full item and device investigation, a data-driven verdict, and an analyst-gated block (section 11).
- **13 read-only commands** covering devices, device inventory, findings, groups, users, remediations, approval requests, runtime

(hardening) policies, Koidex catalog search and risk reports, and fetch-state diagnostics.

- **First Fetch Time Range** parameter to control the initial event-collection lookback window.
- **Tenant Name** parameter and automatic tenant identification: every collected event is stamped with `koi_tenant_name` and `koi_customer_id`, so events from multiple KOI tenants stay distinguishable.
- **Parsing Rule, Modeling Rule, and the Koi Alerts Dashboard** (new XSIAM content).

2. Prerequisites

- An active KOI tenant, and a KOI account with the xt-Administrator role (required to create API keys).
- Cortex XSIAM for event collection, modeling rules, and the dashboard. The commands also work from Cortex XSOAR.
- Outbound HTTPS access from your Cortex tenant/engine to `https://api.prod.koi.security/`.
- **An egress IP that KOI accepts.** KOI applies IP-based restrictions at its edge to the sensitive API endpoints (users, inventory, Koidex, governance). Requests leaving from shared datacenter or cloud egress ranges can be rejected with HTTP 403 even when the API key is valid and the same key succeeds from a corporate network. Event collection is unaffected. If your Cortex tenant egresses from a cloud range, run the KOI integration on a Cortex engine whose egress IP KOI accepts — see section 15.

3. Create Your KOI API Key

1. **Ensure the appropriate role.** To create an API key you must hold the xt-Administrator role in KOI.
2. **Open Settings.** Access the Settings page from the top navigation bar of the KOI console.
3. **Open the API Access tab.** In Settings, select the API Access tab.
4. **Create the key.** Click Create new API key.
5. **Copy the key.** Within a few seconds the new key appears in the table. Click Copy next to it and store it securely — you will paste it into the integration instance.

Treat the API key like a password: store it in a secret manager and rotate it according to your organization's policy.

4. Install the Pack

1. In Cortex XSIAM/XSOAR, open Marketplace.
2. Search for **KOI** and open the pack.
3. Click Install. The integration, parsing/modeling rules, and the dashboard are installed together.

5. Configure the Integration Instance

In Cortex XSIAM, go to Settings → Configurations → Automation & Feed Integrations; in Cortex XSOAR, go to Settings → Integrations → Instances. Search for KOI and click Add instance.

5.1 Connection parameters

Parameter	Description	Default
Server URL	The KOI API server URL.	<code>https://api.prod.koi.security/</code>
API Key	The key created in section 3.	—
Tenant Name	Optional label stamped on every collected event as <code>koi_tenant_name</code> (e.g., "Production"). The tenant UUID (<code>koi_customer_id</code>) is captured automatically from the API key.	—
Trust any certificate	Skip TLS verification (not recommended).	Off
Use system proxy settings	Route via the system proxy.	Off

5.2 Event-collection parameters (Cortex XSIAM only)

Parameter	Description	Default
Fetch events	Enables the event collector.	Off
Fetch event types	Which KOI event streams to collect: Alerts, Audit.	Alerts, Audit
Audit log type filter	Optionally restrict audit collection to selected types (approval_requests, devices, endpoints, extensions, firewall, guardrails, notifications, policies, remediation, requests, settings, vetting).	All types
First Fetch Time Range	Lookback window for the first fetch only (e.g., "3 days", "1 hour"). Afterwards the collector always resumes from the last-fetched event timestamp.	3 days
Maximum number of events per fetch	Cap per event type per fetch cycle.	5000
Events Fetch Interval	How often the collector runs.	1 minute

Multi-tenant tip: to collect from several KOI tenants, create one instance per tenant, each with its own API key and a distinct Tenant Name label — the `koi_tenant_name` field keeps the datasets cleanly separated.

6. Verify the Setup

1. Click Test on the instance. A successful test confirms URL and API key.
2. From the CLI / War Room, run a read-only command:

`!koi-devices-list limit=5`
3. Confirm a device table is returned. If you enabled event collection, allow one fetch interval and check the dataset (section 8).

7. Using the Commands

Commands are grouped below by what they operate on. Commands marked with * modify tenant state; all others are read-only. Most list commands support `page/page_size` for a specific page, or `limit` to auto-paginate.

7.1 Devices & organization

Command	Description
<code>koi-devices-list</code>	List devices registered with KOI (filter by status, last-seen, and more).
<code>koi-device-inventory-get</code>	List the items installed on a single device.
<code>koi-groups-list</code>	List device groups.
<code>koi-users-list</code>	List KOI users.

7.2 Inventory

Command	Description
<code>koi-inventory-list</code>	Paginated list of items installed across your endpoints, with rich filters (marketplace, platform, risk level, publisher, category).
<code>koi-inventory-item-get</code>	Full details for one item by ID + marketplace (+ optional version).
<code>koi-inventory-search</code>	Advanced query-builder search via filter JSON.
<code>koi-inventory-item-endpoints-list</code>	Endpoints that have a specific item installed.

7.3 Governance & policies

Command	Description
<code>koi-policy-list</code>	List all policies.
<code>koi-policy-status-update *</code>	Enable/disable a policy by ID.
<code>koi-runtime-policies-list</code>	List agent runtime (hardening) enforcement policies.
<code>koi-runtime-policy-get</code>	One runtime policy by ID, including its full rule tree.
<code>koi-findings-list</code>	List finding (detection) definitions — resolve finding IDs returned elsewhere.
<code>koi-approval-requests-list</code>	List approval requests.
<code>koi-remediations-list</code>	List remediation suggestions.

7.4 Allowlist & blocklist

Command	Description
koi-allowlist-get / koi-blocklist-get	Retrieve the global allowlist / blocklist.
koi-allowlist-items-add * / koi-blocklist-items-add *	Add one item (item_id + marketplace) or bulk-add from a JSON file entry.
koi-allowlist-items-remove * / koi-blocklist-items-remove *	Remove one item or bulk-remove from a JSON file entry.

7.5 Threat intelligence — Koidex catalog

Command	Description
koi-koidex-search	Search KOI's catalog database by marketplace + search term.
koi-koidex-risk-report	Risk + compliance report for a single catalog item.

7.6 Collector & diagnostics

Command	Description
koi-get-events	Manually pull events. Development/debugging only — keep should_push_events=false to avoid duplicates.
koi-fetch-context-get	Read-only diagnostic: prints the collector's fetch state (last run / high-water mark).
koi-fetch-context-set *	Maintenance: adjust the fetch state to recover a stalled collector.

7.7 Worked examples

List active devices:

```
!koi-devices-list limit=50 status=active
```

Everything installed on one device:

```
!koi-device-inventory-get device_id=DEVICE-123 limit=50
```

Find high-risk items across the fleet:

```
!koi-inventory-search filter_json="{\"field\":\"risk_level\",\"operator\":\"eq\",\"value\":\"high\"}" limit=50
```

Vet a package before approving it:

```
!koi-koidex-search marketplace=npm search_term=axios
!koi-koidex-risk-report item_id=axios marketplace=npm
```

Block a malicious extension organization-wide:

```
!koi-blocklist-items-add item_id=malicious-ext marketplace=chrome_web_store notes="Blocked due to security risk"
```

8. Event Collection into Cortex XSIAM

With Fetch events enabled, the integration collects two streams from KOI on every fetch interval:

- **Alerts** — KOI security alerts (OCSF-shaped Application Security Posture Findings).
- **Audit** — administrative and item activity (installs, updates, policy changes, remediations...).

Collection is incremental: the collector tracks the newest event timestamp per stream (a high-water mark) and resumes from it on every run, so events are neither missed nor duplicated. Events land in the koi_koi_raw dataset, where the pack's parsing rule normalizes them and the modeling rule maps them to XDM.

8.1 Useful XQL starting points

Recent KOI alerts:

```
dataset = koi_koi_raw | filter source_log_type = "Alerts" | sort desc _time | limit 100
```

Audit activity by type:

```
dataset = koi_koi_raw | filter source_log_type = "Audit" | comp count() as events by type | sort desc events
```

Volume per tenant and stream (multi-tenant deployments):

```
dataset = koi_koi_raw | comp count() as events by koi_tenant_name, source_log_type
```

9. KOI Alerts Dashboard

The pack ships the "Koi Alerts Dashboard", which visualizes KOI alert activity from the `koi_koi_raw` dataset. After installing the pack in XSIAM, open Dashboards & Reports, locate Koi Alerts Dashboard, and add it to your dashboards. Data appears once event collection has run (section 8).

10. Koi Unified Script Runner Playbooks

These three playbooks run KOI script packages (for example, the KOI deployment script) on Cortex-Agent endpoints on a recurring schedule. They are designed to be attached to a time-triggered Job and are configured entirely through a JSON List — retargeting or adding scripts never requires editing a playbook.

10.1 Architecture

Playbook	Role
Koi Unified - Script Runner	Job entry point. Loads the "Koi Script Runner" List and loops over its entries — one iteration per entry, each in an isolated context. Closes its own investigation when done.
Koi Unified - Process Config Entry	Per-entry logic: honors the disabled flag, validates the entry (invalid entries are reported in the war room, never silently dropped), invokes the executor, routes the outcome, sends the optional email notification, and cleans up.
Koi Unified - Execute Endpoint Script	Resolves the script name to its Action Center UUID (or uses a provided UUID), targets connected and unisolated endpoints filtered by OS, groups, and hostnames, executes the script with polling, and returns a structured ScriptResult.

10.2 What must already exist on the tenant (not shipped in the pack)

The playbooks reference several objects strictly by name at runtime. None of these are delivered by the pack — they must be created on the tenant before the first run, with names that match the configuration exactly (case and spacing included):

Dependency	Where it lives	Requirement
KOI script package(s)	Action Center → Scripts Library	Upload the KOI deployment script(s) yourself — they are NOT part of this pack. The Library name must match the List's <code>script.name</code> exactly (e.g., "KOI Deployment Script - Windows"). To make the reference rename-proof, pin <code>script.uuid</code> instead.
Endpoint group(s)	Endpoints → Endpoint Groups	Every group named in <code>target.endpoint_groups</code> must exist and contain the intended agents. Alternatively, scope by <code>target.endpoint_hostnames</code> .
Target agents	Endpoints	Cortex agents installed on the targets, <code>CONNECTED</code> and not isolated, with an OS matching the entry's <code>endpoint_os</code> — otherwise the entry reports skipped.
Configuration List	Settings → Object Setup → Lists	A JSON List named exactly "Koi Script Runner" (section 10.3).
Mail-sender instance (optional)	Automation & Feed Integrations	Required only when notifications are configured: an enabled instance that supports send-mail. If <code>notification.sendmail_instance.name</code> is set, an instance with that exact name must exist.

Everything binds by name at run time: renaming a script in Action Center, an endpoint group, the List, or a mail instance — without updating the List configuration — causes the next run to fail or skip (the war-room reason states which reference broke).

10.3 Configuration — the "Koi Script Runner" List

Create a JSON List named exactly "Koi Script Runner" (Settings → Configurations → Object Setup → Lists) containing an array of entries:

```
[
{
  "disabled": false,
  "script": { "name": "KOI Deployment Script - Windows" },
  "target": {
    "endpoint_groups": ["KOI Endpoints"],
    "endpoint_hostnames": [],
    "endpoint_os": "Windows"
  },
  "notification": {
```

```

"sendmail_instance": { "name": "internal-smtp" },
"recipients": ["ops@example.com"]
}
}
]
```

Field	Required	Description
disabled	No	Set true to skip the entry entirely.
script.name	Yes*	Exact script name in the Action Center Script Library. (*Either name or uuid.)
script.uuid	No	Script UUID — takes precedence over name and disambiguates duplicate names.
script.polling_interval_in_seconds	No	Poll cadence while the script runs (default 60).
script.timeout_in_seconds	No	Overall execution timeout (default 1800).
target.endpoint_os	Yes	Windows, macOS, or Linux (case-insensitive).
target.endpoint_groups	One of these	Endpoint group names to scope execution.
target.endpoint_hostnames	One of these	Specific hostnames to scope execution.
notification.recipients	No	Email recipients for the per-entry outcome; omit for no email.
notification.sendmail_instance.name	No	Specific mail-sender instance; omit to use any enabled one.

10.4 How scripts are matched to endpoints

The pairing of script to operating system is declared per entry, and enforced at dispatch time: each entry queries endpoints with the entry's groups/hostnames AND its endpoint_os as a platform filter, restricted to connected, unisolated agents. A Windows entry therefore can never execute on a Mac even when both target the same mixed endpoint group — and one shared group can serve every OS, with each entry picking out its own members.

10.5 Job setup

1. Investigation & Response → Automation → Jobs → New Job.
2. Trigger: **Time triggered**, choose the recurrence (e.g., hourly).
3. Playbook: **Koi Unified - Script Runner**. Save, then use Run now for an immediate first run.

10.6 Run outcomes

Outcome	Meaning	Notification
ok: true (+ action_id)	Script dispatched and completed; action_id links to Action Center.	Success email (when recipients configured).
skipped: true	Entry valid, but no connected endpoints currently match the target scope — nothing to do this run. Logged as an info entry.	None — keeps recurring jobs quiet for not-yet-populated OS scopes.
ok: false + reason	Real failure: script not found, ambiguous name, endpoint query error, or execution failure. The reason states which.	Failure email (when recipients configured).

11. Alert Triage, Investigation & Response Playbooks

A second, independent playbook set turns each KOI alert into an investigated, classified and — when you approve it — actioned case. Attach KOI - Alert Triage to your KOI alerts and the rest runs automatically.

11.1 The playbooks

Playbook	Role
KOI - Alert Triage	Main. Extracts the alert context, runs the item and device investigations, computes a verdict, posts a summary, and routes: auto-close benign, escalate the rest, and open an analyst-gated block for Malicious.
KOI - Extract Alert Context	Sub. Parses the koi_context JSON embedded in the alert description into the KoiContext object.
KOI - Investigate Item	Sub. Full investigation of the flagged item (11.3).
KOI - Investigate Device	Sub. Full investigation of the affected device (11.4).
KOI - Enrich Item	Sub. Lightweight enrichment (catalog risk plus endpoint exposure) for callers that do not need the full investigation.
KOI - Block and Remediate	Investigates the item, skips it if already blocked, then adds it to the organization blocklist only after analyst approval.
KOI - MCP Server Audit	Scheduled hunt. Flags MCP servers at or above a risk threshold; attach to a time-triggered Job.

11.2 Triage flow and verdicts

The verdict is keyed on KOI's own alert type and risk level, with the independent Koidex catalog risk as a corroborating signal:

Verdict	Condition	Action
Malicious	Alert type is Removed from Marketplace, Publisher Compromised or Unvetted MCP Server; or KOI risk level is High or Critical; or the catalog risk is high/critical.	Raise severity, then open an analyst-gated block for the item.
Benign	Alert type is New Item and KOI risk level is Low.	Auto-close the alert as Resolved.
Suspicious	Anything else (safe default).	Leave open for analyst review.

11.3 What the item investigation gathers

- Catalog risk score and level, plus KOI's AI-generated risk summary.
- Your organization's own inventory record for the item: installs count, endpoint count, publisher, signing status and organization risk level.
- The endpoints the item is installed on and the users affected (resolved when the installed version is known).
- Governance state — whether the item already appears on the organization blocklist.
- Remediation and approval history filtered to that item.

11.4 What the device investigation gathers

- Every item installed on the affected device, with version, marketplace, publisher and install path.
- Which of those items are at or above the configured risk level (default: high, critical), posted as a risky-items table.
- Remediations recorded against that host.

11.5 Response safety

Adding an item to the blocklist is the only state-changing action in this set, and it is gated. KOI - Block and Remediate presents the full investigation to an analyst and writes only on approval; the triage always calls it with auto_block set to false, so a Malicious verdict can never block software without a human decision. Setting auto_block to true (only meaningful when you invoke the playbook directly) skips the gate — use with care.

11.6 Attaching the triage

1. Ingest KOI alerts into Cortex (section 8) so each alert carries its koi_context payload.
2. Attach **KOI - Alert Triage** as the playbook for KOI alerts, or run it on an existing alert from the CLI:

```
!setPlaybook playbookId="KOI - Alert Triage"
```

3. Confirm in the war room: an item investigation summary, a device investigation summary, and a triage summary carrying the verdict.

KOI - Investigate Item, KOI - Investigate Device and KOI - Block and Remediate declare required inputs, so they are designed to be called as sub-playbooks (or invoked by an analyst supplying those inputs) rather than attached directly to an alert.

12. Deploying the Full Pack as a Single Object (demisto-sdk)

The pack can be delivered to a tenant as one artifact — a pack zip — using Palo Alto Networks' demisto-sdk. This is the recommended path for dev/test tenants and CI pipelines; production tenants normally install from the Marketplace.

12.1 Choosing an install path

The path you choose determines whether event collection works at all. Commands and playbooks behave the same on every path; the collector and the rules do not.

Path	What lands on the tenant	Events flow?	Use when
Marketplace	Integration, parsing/modeling rules, dashboard and playbooks	Yes	Production tenants
demisto-sdk --xsiam	Everything, at the version in your repo	Yes	Dev/test and CI
Pre-built dist/Koi.zip	Integration plus the 3 Script Runner playbooks only — no rules, no dashboard	No	Not for XSIAM
Manual per-item	Only the items you import yourself	Only if you add the rules	Partial adoption

The pre-built dist/Koi.zip is an XSOAR-marketplace build. Inside it the integration carries `isfetchevents: false` (the `isfetchevents:xsoar: false` override has been resolved into the base field), and the artifact contains no parsing rules, no modeling rules and no dashboard. Uploading it to a Cortex XSIAM tenant produces an instance whose commands all work while the `koi_koi_raw` dataset stays permanently empty — which presents like a collector fault but is simply the wrong artifact. For XSIAM, install from the Marketplace or upload with `demisto-sdk --xsiam`.

12.2 One-time setup

1. **Install the SDK:** `pip3 install demisto-sdk` (1.38+ required — older versions reject packs that declare the platform marketplace).
2. **Place the pack in a content-repo structure.** `demisto-sdk` only runs inside a repo shaped like `demisto/content`: a git repository containing `Packs/Koi/<pack contents>`, an empty `.private-repo-settings` file at the root, and `Tests/secrets_white_list.json` containing `{"ioscs":[],"files":[],"generic_strings":[]}`.
3. **Set the connection environment variables:** `DEMISTO_BASE_URL` (the api- URL of the tenant), `DEMISTO_API_KEY`, and `XSIAM_AUTH_ID` (the key's ID).

12.3 Build and push

Validate, build the single pack artifact, and upload:

```
demisto-sdk validate -i Packs/Koi/Playbooks
demisto-sdk zip-packs -i Packs/Koi -o zipped
demisto-sdk upload -i Packs/Koi -z --xsiam
```

12.4 Notes verified in practice

- **API key type matters.** `demisto-sdk` sends the API key as-is, which works only with a Standard XSIAM API key. With an Advanced key the SDK cannot connect; in that case build the artifact with `zip-packs` and POST zipped/uploadable_packs/Koi.zip (multipart, field name "file") to `https://api-<tenant>/xsoar/contentpacks/installed/upload?skipVerify=true&skipValidation=true` using the advanced-key signed headers. Note the endpoint is under `/xsoar/`, not `/xsoar/public/v1/`.
- **API modules.** The Koi integration imports `ContentClientApiModule`; the build inlines it from `Packs/ApiModules/Scripts/ContentClientApiModule/` — copy that folder from the `demisto/content` repository into your content repo before running `zip-packs`.
- **Pack items become system-owned.** After a pack install, its items (playbooks, integration) are marked system and reject individual item uploads. Ship subsequent changes as a new pack version (bump `currentVersion` in `pack_metadata.json`, add a release note, re-upload the zip).

After upload, the pack appears in the tenant at the new version and every item (integration, playbooks, rules, dashboard) is installed together — see section 14 to validate.

13. Onboarding Individual Items Manually

If you prefer to adopt only part of the pack (or cannot use the SDK), each item can be onboarded by hand in the tenant UI. Import order matters where items reference each other by name.

13.1 Item-by-item reference

Item	How to onboard manually
Integration (Koi.yml)	Marketplace → install the KOI pack (recommended); or Settings → Configurations → Automation & Feed Integrations → Upload Integration and select the YAML.
Playbooks — Script Runner (3 files)	Investigation & Response → Automation → Playbooks → Import. Import in dependency order: 1) Koi Unified - Execute Endpoint Script, 2) Koi Unified - Process Config Entry, 3) Koi Unified - Script Runner — sub-playbook references bind by name.
Playbooks — Triage & response (7 files)	Same import screen. Import the sub-playbooks first (Koi - Extract Alert Context, Koi - Investigate Item, Koi - Investigate Device, Koi - Enrich Item), then Koi - Block and Remediate and Koi - MCP Server Audit, and finally Koi - Alert Triage. References bind by name.
Configuration List	Settings → Configurations → Object Setup → Lists → New List. Name it exactly "Koi Script Runner", type JSON, and paste the configuration array (section 10.3).
Job	Investigation & Response → Automation → Jobs → New Job. Time-triggered, choose the "Koi Unified - Script Runner" playbook, set the recurrence.
Parsing / Modeling Rules	XSIAM: Settings → Data Management → Parsing Rules (and Modeling Rules) → add the content of the pack's .xif files as user-defined rules targeting dataset koi_koi_raw.
Koi Alerts Dashboard	Dashboards & Reports → New Dashboard → Import, select Koi_Alerts_Dashboard.json.

Keep item names unchanged when importing manually — the Job references the main playbook by name, the main playbook references the List by name, and the sub-playbooks reference each other by name.

13.2 Running only the Script Runner Job

A common case is wanting nothing from KOI's API — only the ability to run KOI script packages on Cortex-Agent endpoints on a schedule. That needs a small subset of the pack.

The three Script Runner playbooks invoke no koi-* command at all. They use only the Cortex-native core-get-scripts, core-get-endpoints and core-script-run, plus the built-in send-mail for notifications. Consequently this deployment requires no KOI API key, no KOI integration instance, no parsing or modeling rules and no dashboard — and it works on Cortex XSOAR as well as Cortex XSIAM.

Import exactly these four items, in this order. Sub-playbook references bind by name, so importing a parent before its child leaves the parent pointing at a playbook that does not yet exist:

#	Item	Where
1	Koi Unified - Execute Endpoint Script	Investigation & Response → Automation → Playbooks → Import
2	Koi Unified - Process Config Entry	Investigation & Response → Automation → Playbooks → Import
3	Koi Unified - Script Runner	Investigation & Response → Automation → Playbooks → Import
4	Koi Script Runner (JSON List)	Settings → Configurations → Object Setup → Lists → New List, type JSON

Then create the Job: Investigation & Response → Automation → Jobs → New Job, Time-triggered, playbook "Koi Unified - Script Runner", and set the recurrence.

Three things must already exist on the tenant, referenced by name from the List: the KOI script package in Action Center → Scripts Library (its Library name must equal script.name exactly, or pin script.uuid instead), an endpoint group named in target.endpoint_groups containing connected and unisolated agents whose OS matches target.endpoint_os, and — only if you configure notifications — an enabled mail-sender instance matching notification.sendmail_instance.name.

To validate this deployment on its own, run only test 9 of the test guide: Run now on the Job, then confirm ScriptResult ok:true with an action_id per entry and COMPLETED_SUCCESSFULLY in Action Center. Tests 1 to 8 all exercise the KOI API and do not apply.

14. Validating That Everything Works

Run these checks after any deployment (SDK, Marketplace, or manual). Each row gives the action and the evidence that proves the component is healthy.

Component	How to validate	Expected evidence
API connectivity	Instance → Test button; then run <code>!koi-devices-list limit=5</code> in the CLI.	Test returns success; a device table renders.
Event collector	Wait one fetch interval, then run the XQL query below (a).	Recent rows with <code>source_log_type</code> Alerts/Audit; count grows between runs.
Parsing & modeling rules	Inspect the XQL result fields.	Fields like <code>source_log_type</code> , <code>alert_type</code> , <code>severity</code> , <code>mcp_*</code> are populated (not only <code>_raw_log</code>).
Script Runner playbooks	Jobs → select the job → Run now; open the run.	Work Plan is all green; war room shows <code>ScriptResult ok:true</code> with an <code>action_id</code> per executed entry, and SKIPPED info entries for OS scopes with no endpoints.
Endpoint execution	Action Center → All Actions → find the <code>action_id</code> .	Status <code>COMPLETED_SUCCESSFULLY</code> on the expected endpoints.
Notifications	Configure a mail-sender instance; re-run the job.	Success/failure email arrives at the List's recipients; no send-mail errors in the war room.
Alert triage playbooks	Run KOI - Alert Triage on a KOI alert (attach it, or <code>!setPlaybook</code>).	War room shows an item investigation summary, a device investigation summary, and a triage summary carrying the verdict; benign alerts auto-close, others escalate.
Analyst-gated block	Let a Malicious verdict reach the block step.	The run parks at <code>runStatus "waiting"</code> on the approval task and <code>koi-blocklist-items-add</code> has NOT executed. It runs only after an analyst approves.
Dashboard	Open Koi Alerts Dashboard.	Widgets render data after the collector has ingested alerts.

(a) Collector validation query:

```
dataset = koi_koi_raw | sort desc _time | limit 10
dataset = koi_koi_raw | comp count() as events by source_log_type
```

Failure-path checks worth running once: temporarily set "disabled": true on a List entry (that entry must be skipped silently), and misspell a script name (the run must report a clear "not found" reason rather than hang).

15. Troubleshooting

Symptom	Likely cause	Resolution
"Authorization Error: Verify your API Key"	Key invalid, revoked, or pasted with whitespace	Re-create the key (Settings → API Access, xt-Administrator role) and update the instance.
Test succeeds but no events arrive	Fetch events disabled; wrong event types; or a stalled fetch state	Enable Fetch events and check Fetch event types. Run <code>koi-fetch-context-get</code> to inspect the high-water mark; if it is stuck (e.g., in the future), recover with <code>koi-fetch-context-set</code> .
koi-inventory-search returns a 400 error	<code>filter_json</code> is not a valid query-builder object	Provide filter JSON in the documented structure (field/operator/value rules); see the command reference for examples.
Koidex commands return a 400 error	Missing required arguments	<code>koi-koidex-search</code> requires both <code>marketplace</code> and <code>search_term</code> ; <code>koi-koidex-risk-report</code> requires <code>item_id</code> and <code>marketplace</code> .
Duplicate events after manual pulls	<code>koi-get-events</code> was run with <code>should_push_events=true</code>	Use <code>koi-get-events</code> for debugging only, and keep <code>should_push_events=false</code> .
HTTP 429 / rate limiting	Fetch volume too aggressive	Lower Maximum number of events per fetch or increase the fetch interval.
HTTP 403 on inventory, users, Koidex or governance commands, while event collection keeps working	KOI's edge is rejecting the source IP — typically a shared datacenter or cloud egress range	Run the integration on a Cortex engine whose egress IP KOI accepts (for example a corporate/office network), or ask KOI to allowlist your egress. Confirm with <code>!koi-users-list limit=1</code> — the same key succeeds from an accepted IP and returns 403 from a rejected one.

16. Support & References

- KOI product documentation: <https://docs.koi.ai>
- Pack release notes: see the KOI pack page in the Cortex Marketplace.

- For issues with the integration content, contact your Palo Alto Networks / KOI support channel.